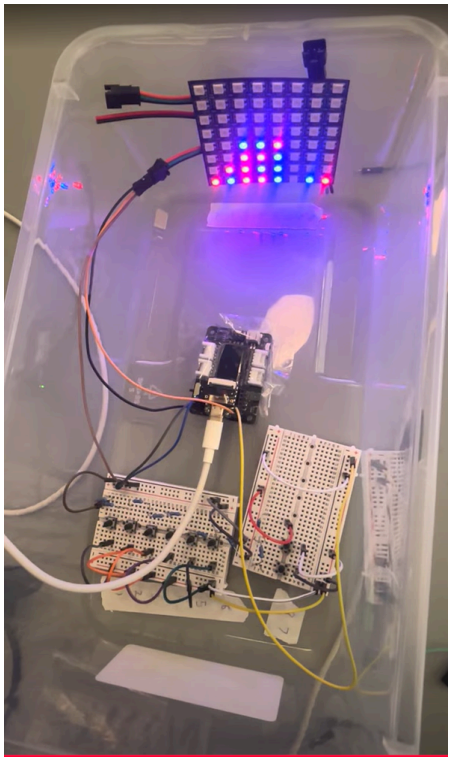
Connect 4 AI Player: Embedded Computer Vision and Game Strategy on ESP32

Team Members: Nathan Zhang and Kristian Prifti

Course: ESE 3600: TinyML

Abstract

This project describes a fully automatic Connect 4 system that lets a human play against an AI on a real, physical board. Everything runs on a single microcontroller, no laptop or cloud connection required. Using a Seeed Studio XIAO ESP32-S3, the system takes a picture of the board with its camera module and uses a small Convolutional Neural Network (CNN) to recognize where the pieces are. After detecting the game state, a Minimax algorithm with Alpha-Beta pruning chooses the best move and shows it on the LED board. This report explains how the entire system was built and shows that both computer vision and game-search algorithms can run together on a small embedded device.



1. Introduction

Connect 4 is a classic solved strategy game that has long served as a benchmark for algorithmic game theory. While digital versions are ubiquitous, physical implementations that include an intelligent opponent often rely on manual input or complex, fragile sensor arrays like Hall effect sensors beneath every cell. Our project seeks to modernize this physical interaction by closing the feedback loop using computer vision.

Our system includes an 8x8 RGB LED matrix (bigger than the usual 6x7 board), buttons for human input, and an ESP32-S3 board with a camera. The key idea is that the

microcontroller takes a picture of the board, cuts it into tiles, and classifies each one as Empty, Red, or Blue. This allows the AI to “see” the game just like a human would.

2. Motivation

2.1 Physical Play Without Screens

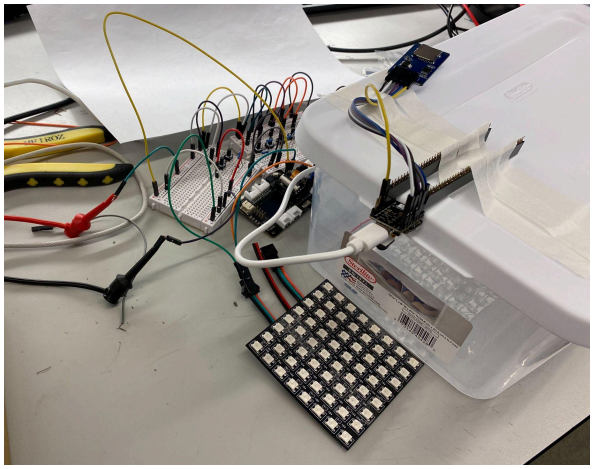
Many people are tired of screen-based games. We wanted to create a hands-on tabletop game that still has a smart computer opponent, without the need for a phone or computer. Connect 4 is simple, interactive, and ideal for this. Because everything runs on the ESP32, the system also uses very little power.

2.2 Proving Edge AI Capabilities

We also wanted to show what modern microcontrollers can do. By processing images directly on the ESP32-S3, we avoid cloud delays. Running both a computer vision model and a game-search algorithm on such a small chip pushes the limits of TinyML and demonstrates practical edge-AI performance.

3. Dataset Details

3.1 Data Collection



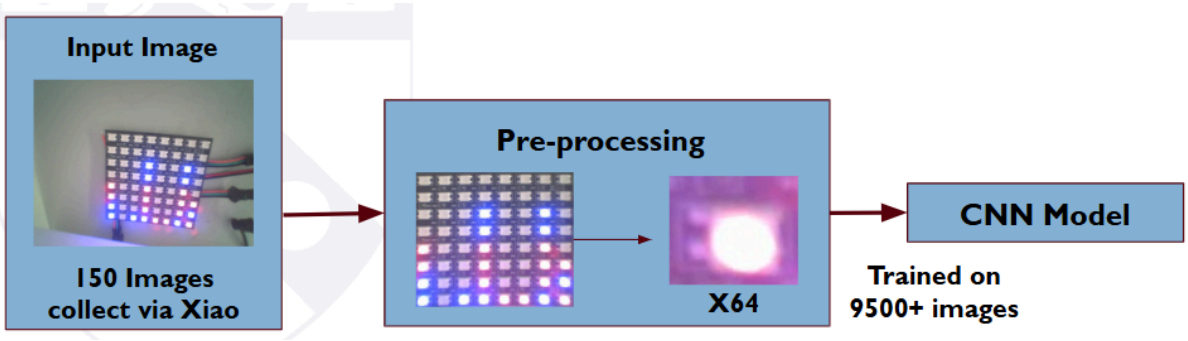
Due to the lighting of the led-matrix we created our own dataset. Using the ESP32’s camera, we captured 150, 1600x1200 images of the board. A script automatically lit the board with random patterns of a Connect 4 board state and captured images that matched the exact conditions the AI would see. Using the ESP32 camera module to collect the dataset ensured that the training data perfectly matched the perspective and lighting conditions the model would encounter during deployment.

3.2 Preprocessing

The raw board images required significant preprocessing to be suitable for a lightweight CNN. First we tried to detect the whole board at once but the accuracy of the model was

not good. We believe we did not have enough data points for the complex task of correctly labeling 64 leds together. We changed our approach to a tiled based one, where the model would now have to classify each led separately and then we reconstruct a board state.

Preprocessing included finding the corners of the led matrix and cropping each data point. Then we segmented each cropped picture into 64 smaller ones, one for each led. We ended up with 9,500+ labeled tile images. These tiles were resized to a standard 32x32 pixel resolution to match the input tensor requirements of our model design. We used these tile images to categorize them into three distinct classes: Class 0 for Empty (LED Off), Class 1 for Red (Human Player), and Class 2 for Blue (AI Player). The resulting dataset was well-balanced, with 4,864 empty samples, 2,342 red samples, and 2,330 blue samples, preventing class bias during training.



4. Model Choice and Design

4.1 Architecture Selection

We chose a small Convolutional Neural Network because CNNs handle images well and can run efficiently on microcontrollers. A larger model like MobileNet would be too big, while a fully dense network would not capture spatial patterns.

Our network includes:

- two 3×3 convolution layers (16 and 32 filters)
- max pooling after each
- one depthwise separable convolution layer to reduce computation
- global average pooling to reduce memory use
- a 64-neuron dense layer
- a 3-class softmax output

This architecture fits easily within ESP32 limits while remaining accurate.

4.2 Quantization

We converted the model to TensorFlow Lite with 8-bit dynamic range quantization. This reduced the model size from 305 KB to about 127 KB and sped up inference without hurting accuracy.

5. Model Training and Evaluation Results

5.1 Training Process

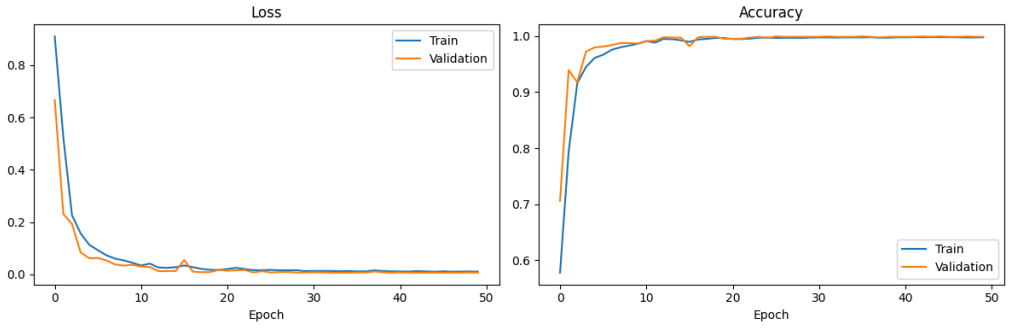
We trained the model in Google Colab using an 85/15 train-test split and the Adam optimizer with a learning rate of 0.001. Early stopping and learning-rate reduction helped avoid overfitting. The model converged in about 50 epochs.

5.2 Evaluation Metrics

The evaluation results on the held-out validation set were exceptional. The model achieved a final validation accuracy of 99.86% and a validation loss of 0.0065. Breaking this down by class reveals the model's robustness:

- Empty Tiles: 99.7% accuracy (730 samples)
- Red Tiles: 100.0% accuracy (351 samples)
- Blue Tiles: 100.0% accuracy (350 samples)

Qualitative verification was performed by visualizing random samples from the validation set alongside their predicted labels. The model successfully distinguished between the Red and Blue LEDs despite variations in brightness and successfully identified "Empty" slots even when adjacent LEDs caused minor light bleeding.



5.3 On-Device Performance

The EloquentTinyML library loaded the quantized model into a 100 KB tensor arena. For each turn, the system captures one frame, splits it into 64 tiles, and runs the CNN on each tile. Although running all 64 inferences took about 13 seconds, the game was still

functional and playable. We would like to improve this in the future by reducing the total inference time, either by optimizing the model further or by using faster image-processing methods.

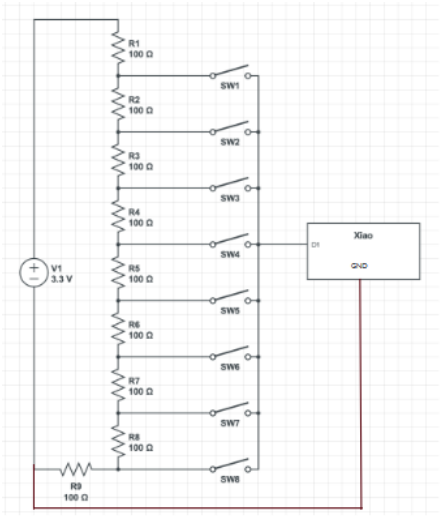
6. Deployment and Hardware Details

6.1 Hardware Implementation

The system uses:

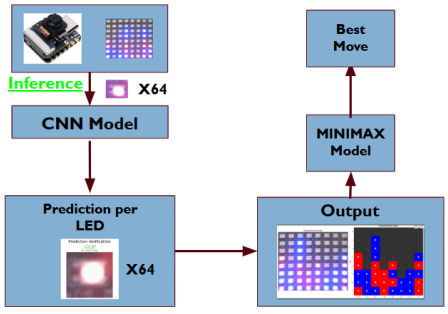
- Seeed Studio XIAO ESP32-S3 Sense with built-in OV2640 camera
- 8×8 WS2812B LED matrix
- 8 push buttons for column input

Because we did not have enough pins to use one for each button input, we read all 8 buttons through one analog pin using a voltage-divider ladder. By arranging the buttons in a resistive ladder, we multiplexed all 8 inputs into a single Analog Input pin (GPIO D1). The software reads the analog voltage value and maps specific ranges to column indices



6.2 Software and AI Strategy

The software stack is written in C++ using the Arduino framework. After the model predicts the state of each led using the CNN as described above, a whole state of the board is constructed. That state is imputed into a Minimax enhanced with Alpha-Beta pruning algorithm that predicts the best move for the AI.



The search depth is set to 5 turns ahead, balancing intelligence with calculation speed. The evaluation function uses a positional weight matrix to prioritize the center columns, which statistically offer more winning opportunities, and scans the board for 4-cell "windows" to score potential threats and opportunities. To further minimize latency, the AI performs an "instant reaction" check for immediate wins or forced blocking moves before entering the recursive search. This combination of efficient vision and optimized game theory results in an opponent that is both responsive and challenging to beat.

```
const int positionWeights[8][8] = {
    {3, 4,  5,  7,  7,  5,  4, 3}, // Row 0 (Bottom)
    {4, 6,  8, 10, 10,  8,  6, 4}, // Row 1
    {5, 8, 11, 13, 13, 11,  8, 5}, // Row 2
    {5, 8, 11, 13, 13, 11,  8, 5}, // Row 3
    {4, 6,  8, 10, 10,  8,  6, 4}, // Row 4
    {3, 4,  5,  7,  7,  5,  4, 3}, // Row 5
    {2, 3,  4,  5,  5,  4,  3, 2}, // Row 6
    {1, 2,  3,  4,  4,  3,  2, 1}  // Row 7 (Top)
};
```

7. Challenges and Future Work

We faced several challenges during development, including collecting high-quality training data due to LED light bleed and dealing with the ESP32-S3's limited GPIO pins, which we overcame by building a voltage-divider button interface. Looking forward, we aim to improve inference speed, enhance vision robustness under different lighting conditions, and explore replacing the Minimax algorithm with a Deep Reinforcement Learning approach. We also plan to expand the user experience with sound effects or voice feedback and move the hardware off the breadboard onto a perfboard or custom PCB to create a more compact and polished system.

8. Conclusion

We built a complete Connect 4 robot that can visually detect the board, think about its moves, and play against a human, all on a small microcontroller. With over 99% vision accuracy and a response time of only a few seconds, the system demonstrates that meaningful computer vision and game-strategy reasoning can run efficiently on low-power embedded hardware. This project highlights the growing potential of TinyML for real-time, interactive systems and shows that even resource-constrained devices can deliver intelligent, engaging gameplay experiences.

9. Demo Video

https://youtu.be/cvaqpiKe06Y